



Cofinancé par
l'Union européenne



TP6 CDA PRF · RNCP 37873 — CONCEPTEUR
DÉVELOPPEUR D'APPLICATIONS (TP6 CDA)

COURS HTML5, CSS3 & BOOTSTRAP

HTML/CSS/Bootstrap

J3 — Bootstrap + TP intégratif

PROMO 2026-D03, CDA JS FULLSTACK IA

Formateur : Robin Hotton, rhotton@diginamic.fr

Formateur Robin HOTTON

Promotion 2026-D03

Version 1.2 — 2026

Agenda du jour

Matin — Bootstrap

XIV — Intro & 3 piliers

XV — Grille + utilities

XVI — Composants

 **Bloc DigiShop Bootstrap**

Après-midi — Choix + TP final

XVII — Personnalisation

XVIII — CSS vs Bootstrap vs Tailwind

 **TP sommatif /20** — Atelier Céramique

 **QCM final** (20 Q)

À la fin de la journée, vous saurez :

- ✓ Construire un site complet avec Bootstrap en quelques heures
- ✓ Personnaliser Bootstrap sans recompiler Sass
- ✓ Choisir entre CSS custom, Bootstrap et Tailwind
- ✓ Livrer un site responsive et accessible noté /20

XIV — Intro Bootstrap

Pourquoi un framework CSS ?

D'après vous...

Q1 — Quels sont les **3 piliers** de Bootstrap ?

GRILLE + UTILITIES + COMPOSANTS

Grille 12 colonnes — `col-md-6`

Utilities — `.p-3` , `.bg-primary` , `.d-flex`

Composants pré-stylés — navbar, cards, modals

Q2 — Bootstrap a un **look reconnaissable** — **pour quel site** est-ce un inconvénient (ou non) ?

PAS UN PROBLÈME

Dashboard admin (Linear, Stripe)

Outil interne / SaaS B2B

Prototype, MVP rapide

E-commerce standard

VRAI INCONVÉNIENT

Vitrine agence créative

Portfolio designer

E-commerce premium / luxe

Site événementiel signature

Installation CDN — 30 secondes

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mon site Bootstrap</title>

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <button class="btn btn-primary">Hello Bootstrap !</button>

  <!-- Bootstrap JS (juste avant </body>) -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Exercice — Premiers composants Bootstrap

EXERCICE `EX/14\_INTRO-BOOTSTRAP.MD` · ● 5 MIN + ● 15 MIN

Prendre le réflexe **doc Bootstrap** → **copier-coller** → **adapter** — 80 % de l'efficacité du framework tient là.

Sur le squelette CDN, ajouter :

6 boutons colorés — `primary` , `secondary` , `success` , `danger` , `warning` , `info`

Variantes — `btn-outline-*` , `btn-lg` , `btn-sm`

Une icône Bootstrap Icons (CDN séparé)

Référence ouverte en parallèle : getbootstrap.com/docs/5.3 → section "Buttons".

XV — Grille + utilities

Les classes qui font 80 % du travail

D'après vous...

Q1 — Vous voyez `class="col-12 col-md-6 col-lg-4"` sur un bloc. Qu'est-ce que ça veut dire ?

◆ MOBILE-FIRST, PROGRESSIVEMENT

`col-12` mobile : **pleine largeur**

`col-md-6` ≥ 768px : **2 colonnes**

`col-lg-4` ≥ 992px : **3 colonnes**

Q2 — Quelle utility pour `padding: 1rem` ?

❗ .P-3 = PADDING 1REM

Syntaxe : `{p|m}{côté}-{0→5}`

Valeur : `0` = 0 · `1` = 4px · `2` = 8px · `3` = **16px** · `4` = 24px · `5` = 48px

Côté : `t` top · `b` bottom · `s` left · `e` right · `x` horizontal · `y` vertical · *rien* = partout

Exemples : `.pt-2` · `.mx-auto` · `.m-0`

Q3 — Pousser un menu à droite dans une navbar Flex ?

Système de grille — **container** / **row** / **col**

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6">Bloc gauche</div>
    <div class="col-12 col-md-6">Bloc droit</div>
  </div>
</div>
```

◆ CONTAINER

Largeur max + padding latéral.

Réactif aux breakpoints. **container-fluid** pour pleine largeur.

ⓘ ROW

Flex horizontal + gestion des **gutters** (espaces entre colonnes).

◆ COL-*

col-1 à **col-12** = 1/12 à 12/12.

Combinables avec **col-md-***, **col-lg-***.

Top utilities à connaître

Affichage + alignement

d-flex · d-grid · d-none
justify-content-* · align-items-*
gap-3 · text-center · text-end

Espacements

p-3 · m-2 · py-4 · mx-auto
pt pb ps pe (axes)
px py (paires)

Couleurs + visuel

bg-primary · text-light
rounded · rounded-circle
shadow · shadow-lg · border

Tailles + responsive

w-100 · h-100 · vh-100
d-md-none (cacher sur tablette+)
d-lg-flex (afficher en flex desktop)

Exercice — Grille Bootstrap

EXERCICE `EX/15\_GRID-RESPONSIVE.MD` • ● 15 MIN + ● 25 MIN

Construire un layout responsive sans écrire **une seule media query** — uniquement le système `container > row > col-*` et les utilities.

Ligne 1 — 4 cartes statistiques (1 col mobile / 2 tablette / 4 desktop)

Ligne 2 — 2 blocs `col-12` mobile / `col-md-6` + `col-md-6`

Utilities — padding, marges, couleurs (`p-*` , `m-*` , `bg-*`)

Test : redimensionner DevTools de 320 → 1440px, la grille doit basculer aux breakpoints `sm` , `md` , `lg` .

XVI — Composants Bootstrap

Navbar, cards, modals, forms

D'après vous...

Q1 — D'après vous, qu'est-ce qu'un **menu burger** ? Pourquoi ce nom ?

❗ ICÔNE ≡ — 3 TRAITS SUPERPOSÉS

Bouton à **3 traits horizontaux** qui rappelle un burger 🍔. Convention UX mobile : cacher le menu de navigation derrière un clic pour gagner de la place sur petits écrans.

Q2 — Votre menu burger Bootstrap ne s'ouvre pas au clic. Que manque-t-il ?

⚠ LE JS BOOTSTRAP

```
<script src="...bootstrap.bundle.min.js"></script>
```

À placer **juste avant** `</body>`. Sans lui : modals, dropdowns, toasts, collapse → tous statiques. **Piège n°1** du débutant Bootstrap.

Q3 — Méthode la **plus productive** pour utiliser un composant Bootstrap inconnu ?

💎 COPIER L'EXEMPLE DE LA DOC OFFICIELLE

getbootstrap.com/docs/5.3 → composant → copier l'exemple → adapter texte + couleurs. **80 % de la productivité Bootstrap** vient de ce réflexe.

À **bookmarker** dans tous les navigateurs de la promo.

Navbar responsive — code complet

```
<nav class="navbar navbar-expand-lg bg-light">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">Logo</a>
    <button class="navbar-toggler" type="button"
      data-bs-toggle="collapse" data-bs-target="#nav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="nav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#">Accueil</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>
```

Composants les plus utiles

Structure

Navbar — `navbar navbar-expand-lg`

Card — `card > card-body > card-title + card-text`

Container — `container` / `container-fluid`

Formulaires

Inputs — `form-control`

Labels — `form-label`

Select — `form-select`

Checks — `form-check-input`

Feedback

Alerts — `alert alert-success/danger/warning`

Badges — `badge bg-primary`

Toasts — notifications temporaires

Interactions JS

Modals — `data-bs-toggle="modal"`

Dropdowns — `data-bs-toggle="dropdown"`

Tooltips — init manuel JS

Exercice — Navbar + composants Bootstrap

EXERCICE `EX/16\_NAVBAR-COMPOSANTS.MD` • ● 20 MIN + ● 30 MIN

- Navbar responsive copiée-collée de la doc — observer la dépendance au bundle JS (tester sans → le burger reste muet).
- Enrichir : dropdown « Compte », recherche, **modale** centrée, breadcrumb, alert **success** dismissible. Contrainte : **zéro JS custom**, tout par **data-bs-***.
- *(facultatif)* Offcanvas, toast API JS, accordéon, carousel, tooltips initialisés en JS.

XVIII — Sass & Tailwind

Au-delà de Bootstrap : préprocesseur historique et utility-first moderne

Sass / SCSS — le préprocesseur historique

❶ PRÉPROCESSEUR CSS

SCSS (alias **Sass**) est un **surensemble de CSS** qui ajoute variables, nesting, mixins, fonctions, imports — compilé en CSS standard avant livraison au navigateur.

```
$brand: #c9501a;
$radius: 12px;
.boite {
  background: $brand;
  border-radius: $radius;
  &:hover { background: darken($brand, 10%); } // nesting + parent
  .titre { font-weight: bold; } // sélecteur imbriqué
}
```

Pourquoi ça a régné

Variables avant les CSS custom properties

Nesting avant `&` natif

Mixins, fonctions, `@import` partials

Stack standard mid-2010s (Bootstrap 4 en Sass)

Pourquoi ça décline

CSS a rattrapé : `var(--x)`, nesting natif, `@layer`

Build supplémentaire à maintenir

Bootstrap 5 dispo via CDN sans recompiler

Tailwind couvre la factorisation autrement

Tailwind — utility-first, #1 depuis 2024

UTILITY-FIRST

Au lieu d'écrire du CSS et d'inventer des noms de classes, on **compose** chaque élément en posant des **utility classes** atomiques directement dans le HTML.

```
<script src="https://cdn.tailwindcss.com"></script>
```

```
<button class="px-4 py-2 bg-blue-600 text-white rounded-lg hover:bg-blue-700">Envoyer</button>
```

```
<button class="px-4 py-2 border-2 border-blue-600 text-blue-600 rounded-lg hover:bg-blue-50">Annuler</button>
```

```
<button class="px-6 py-3 bg-red-600 text-white rounded-full shadow-lg hover:scale-105 transition">Supprimer</button>
```

Position 2024-26

npm #1 des frameworks CSS depuis 2024

GitHub, Netflix, OpenAI, Vercel

v4 — moteur Rust, build instantané

Combo **React + Tailwind + shadcn/ui**

Pourquoi ça a gagné

Plus de **noms de classes à inventer**

Aucun CSS à maintenir, HTML = source

Tree-shaking auto → bundle minuscule

IA-friendly (Copilot le maîtrise)

TP — Site vitrine intégratif

Sommatif unique du cours, noté /20 — Atelier Céramique Lyon

TP — Brief

SOMMATIF /20 · 3H CHRONO · `TP/TP.MD` — ATELIER CÉRAMIQUE LYON

Un **atelier artisanal de céramique** veut un site vitrine 3 pages au look **chaleureux** (terracotta, ocre, blanc cassé — pas du bleu corporate). 3 h pour livrer.

Livrables — 3 pages + assets

`index.html` — hero + valeurs + galerie + CTA

`galerie.html` — 4-6 vignettes responsive

`contact.html` — formulaire validé HTML5

`assets/css/custom.css` & `README.md`

Exigences à valider

Implémentation libre (CSS, Bootstrap, mix)

HTML sémantique + W3C 0 erreur

Responsive 320 / 768 / 1200 px

Lighthouse Accessibilité ≥ 90

≥ 3 variables CSS dans `:root`

Conclusion — 3 jours **HTML/CSS/Bootstrap**

À retenir + cap sur JavaScript

À retenir — Les 3 jours

HTML

Sémantique > soupe de `<div>`

Accessibilité = design, pas option

Attributs = sécurité + perf + a11y

Formulaires avec `<label>` toujours

CSS

`box-sizing: border-box` reset

Flexbox 1D + Grid 2D

Mobile-first + `min-width`

Variables CSS = maintenabilité

◆ CHOIX D'OUTIL

Bootstrap → productivité standardisée · **CSS custom** → liberté créative · **Tailwind** → entre les deux

Choisir selon le contexte projet, justifier dans le README.



Cofinancé par
l'Union européenne



TP6 CDA PRF · RNCP 37873 — CONCEPTEUR
DÉVELOPPEUR D'APPLICATIONS (TP6 CDA)

COURS HTML5, CSS3 & BOOTSTRAP

Merci !

À LA SEMAINE PROCHAINE POUR LES 5 JOURS DE JAVASCRIPT

Questions, retours, demandes de rattrapage — c'est le moment.

Robin Hotton · rhotton@diginamic.fr

Formateur Robin HOTTON

Promotion 2026-D03

Version 1.2 — 2026